eJPT CTF-1 Walkthrough

Introduction

Welcome! In this walkthrough, I'll explain how I approached and solved the eJPT CTF-1 challenge and successfully collected all the flags.

To encourage learning and hands-on practice, I will not be sharing the actual flag values. Instead, I'll focus on the methodology, tools, and commands used to discover them.

Flag 1: Identifying Search Engine Restrictions

Challenge Hint

> *This tells search engines what to avoid and what not to crawl.*

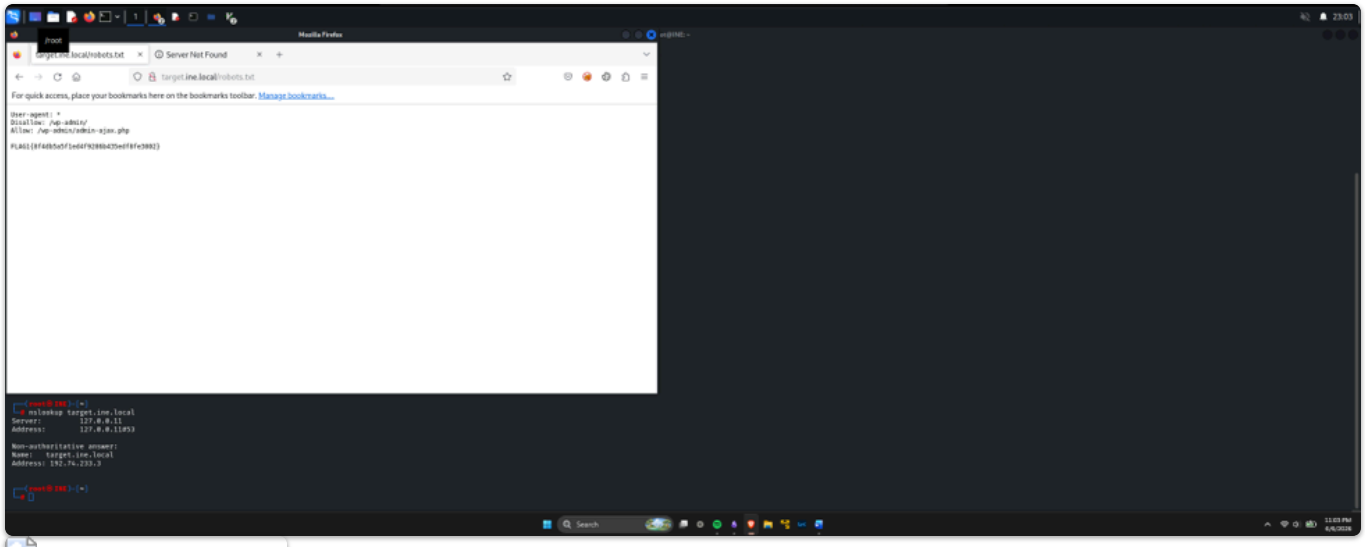
The hint directly points to the robots.txt file. This file is commonly used by website administrators to instruct search engines which directories or files should not be indexed.

Simply append `/robots.txt` to the target URL:

```
http://target.ine.local/robots.txt
```

Reviewing the contents of this file reveals the location needed to obtain the first flag.

Screenshot



Flag 1 Screenshot

Flag 2: Identifying the Website and Version

Challenge Hint

> *What website is running on the target, and what is its version?*

For this flag, basic service enumeration is sufficient. Running an Nmap scan with service detection and version enumeration provides the required information.

Command Used

```
nmap -sCV -A -O target.ine.local
```

Explanation

- `-sC` runs default NSE scripts.
- `-sV` detects service versions.
- `-A` enables OS detection, version detection, script scanning, and traceroute.
- `-O` attempts operating system detection.

The scan output reveals the web application and its version, which corresponds to the second flag.

Screenshot


Flag 2 Screenshot

Flag 3: Discovering Hidden Directories

Challenge Hint

> *Directory browsing might reveal where files are stored.*

This flag requires directory enumeration. Any directory brute-forcing tool can be used, including:

- Dirb
- FFUF
- DirBuster

- Gobuster

Example Command

```
dirb http://target.ine.local
```

While reviewing the results, several directories may appear interesting. Some may lead to login pages or inaccessible locations that do not contribute to the challenge.

However, pay close attention to paths discovered by the scanner, even if direct access initially appears restricted.

One of the discovered paths is:

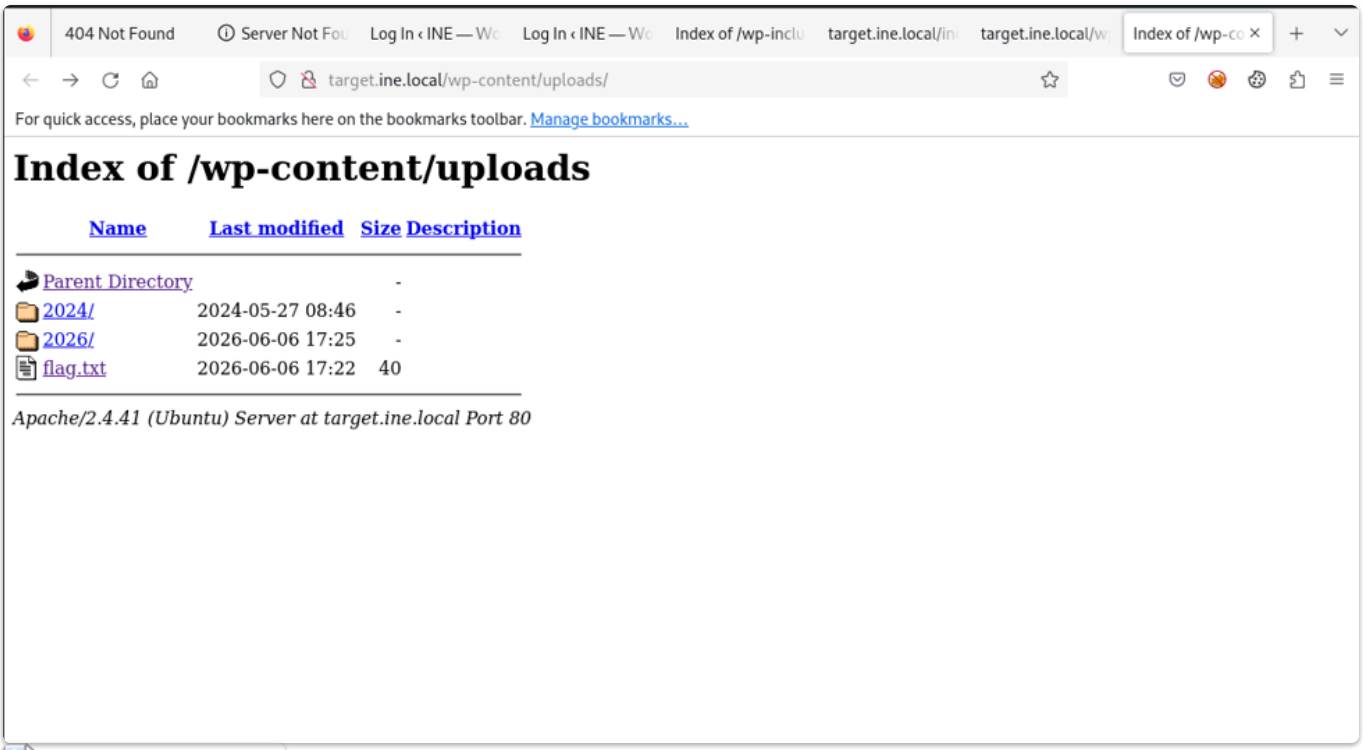
```
/wp-content/uploads/
```

Navigating to:

```
http://target.ine.local/wp-content/uploads/
```

reveals the location containing the third flag.

Screenshot



Flag 3 Screenshot

Flag 4: Finding an Exposed Backup File

Challenge Hint

> *An overlooked backup file in the webroot can be problematic if it reveals sensitive configuration details.*

This flag is largely based on understanding common WordPress misconfigurations.

The hint suggests the presence of a backup file that has been accidentally left accessible on the web server.

Researching common WordPress backup file locations can help identify the correct path. Accessing the location downloads a file with a `.bak` extension.

After downloading the file, inspect its contents using:

```
cat filename.bak
```

The contents reveal the fourth flag.

Screenshot

```
┌──(root㉿INE)-[~/reports/http_target.ine.local]
└─# gobuster dir -u http://target.ine.local -w /usr/share/wordlists/dirb/common.txt -x .bak
===============================================================
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://target.ine.local
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.6
[+] Extensions:              bak
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
/.hta                (Status: 403) [Size: 281]
/.htaccess           (Status: 403) [Size: 281]
/.htpasswd           (Status: 403) [Size: 281]
/.htpasswd.bak       (Status: 403) [Size: 281]
/.htaccess.bak       (Status: 403) [Size: 281]
/.hta.bak            (Status: 403) [Size: 281]
/index.php           (Status: 301) [Size: 0] [--> http://target.ine.local/]
/robots.txt          (Status: 200) [Size: 108]
/server-status       (Status: 403) [Size: 281]
/wp-admin            (Status: 301) [Size: 323] [--> http://target.ine.local/wp-admin/]
/wp-config.bak       (Status: 200) [Size: 3438]
/wp-content          (Status: 301) [Size: 325] [--> http://target.ine.local/wp-content/]
/wp-includes         (Status: 301) [Size: 326] [--> http://target.ine.local/wp-includes/]
Progress: 9228 / 9230 (99.98%)
/xmlrpc.php          (Status: 405) [Size: 42]
===============================================================
Finished
===============================================================
```

Backup File Discovery

```
┌──(root㉿INE)-[~/reports/http_target.ine.local]
└─# curl -O target.ine.local/wp-config.bak
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  3438  100  3438    0     0  1639k      0 --:--:-- --:--:-- --:--:-- 3357k

┌──(root㉿INE)-[~/reports/http_target.ine.local]
└─# cat wp-config.bak
```

Screenshot

```
 * @package WordPress
 */

// ** Database settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
define( 'DB_NAME', 'test' );

/** Database username */
define( 'DB_USER', 'test' );

/** Database password */
define( 'DB_PASSWORD', 'test' );

/** Database hostname */
define( 'DB_HOST', 'localhost' );

/** Database charset to use in creating database tables. */
define( 'DB_CHARSET', 'utf8' );

/** The database collate type. Don't change this if in doubt. */
define( 'DB_COLLATE', '' );

/**#@+
 * Authentication unique keys and salts.
 *
 * Change these to different unique phrases! You can generate these using
 * the {@link https://api.wordpress.org/secret-key/1.1/salt/ WordPress.org secret-key service}.
 *
 * You can change these at any point in time to invalidate all existing cookies.
 * This will force all users to have to log in again.
 *
 * @since 2.6.0
 */
define('AUTH_KEY',         '}Mq^#|v{n0fQ6Vn[tr 6e4glzi:OVs/9(IQ .7f^dp3ym4,th-O$Qx.]|2+(t(sE');
define('SECURE_AUTH_KEY',  'S_LKQ#*}p*U}kdX[GNNVM2*0YISNQ&zrFl jEUNq5T}0Zg|,sO|yB68^|N*1nS-p');
define('LOGGED_IN_KEY',    '`tz-Uz9Ixka,5z0J BD0l/zfU|r2|;9BGL5l~A1RQtZMwh=JftaU$2)$FI%v};|E');
define('NONCE_KEY',        '▷ZN961k>aHWJ*R8#&x+rR>3g|<[:G 8B+rqPH WrWet1SC60+ LL/S+=[G-&g7)');
define('AUTH_SALT',        '+<2l=;osCL(L)zV[=uvr[}2^j-16(gFq18V<m|fP<R{7DV`^=O&bb3fxY+Jf|-;C');
define('SECURE_AUTH_SALT', 'HG&/Q/ceR-$;?jCL}<cL4@LKzDjv,M=K-gR<]iHiAqcHQO+rXcWn/jMt0#K,uWq%');
define('LOGGED_IN_SALT',   'REsFv+OsL*qd=yV<oPaAXeYj@f)A[/Wm5-?|_4d::(;dXcps`rgJf]t4B0Q3)RcH');
define('NONCE_SALT',       ' Q.:O=pFDTA-lNBe%kjJu(mp7$cQrF|IZ _hOWDA&Q18w6CL(<{+1$a-ZJ~<(!_;');


/** FLAG4{92658b2921e2496da66702ee902c1de0} */


/**#@-*/

/**
 * WordPress database table prefix.
 *
 * You can have multiple installations in one database if you give each
```

Flag 4 Screenshot

Flag 5: Website Mirroring and Source Analysis

Challenge Hint

> *Certain files may reveal something interesting when mirrored.*

The keyword "mirrored" strongly suggests the use of a website mirroring tool such as HTTrack.

Command Used

```
httrack http://target.ine.local -O INE
```

This command creates a local copy of the target website.

Once the website has been mirrored, navigate to the output directory and search through the downloaded files.

Since previous flags follow a consistent naming convention, searching recursively for the fifth flag is straightforward.

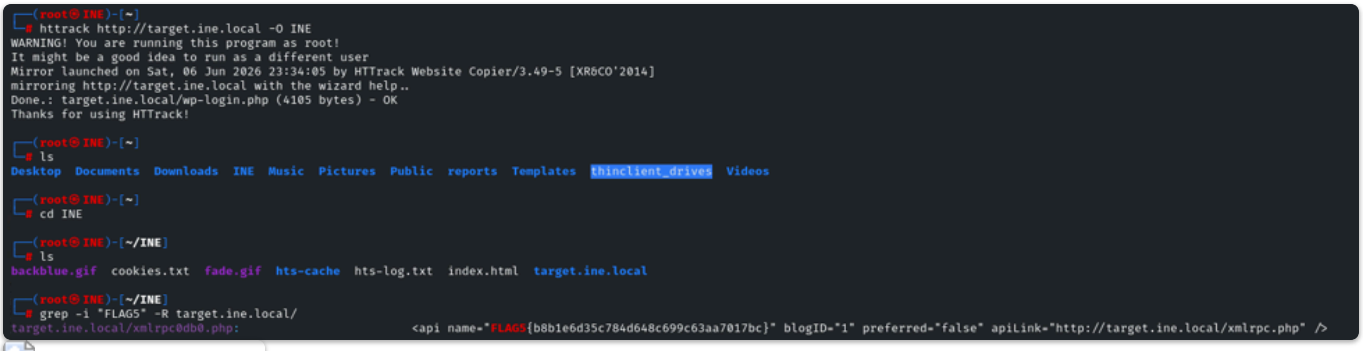
Command Used

```
grep -i "FLAG5" -R target.ine.local/
```

This command recursively searches all mirrored files for occurrences of "FLAG5".

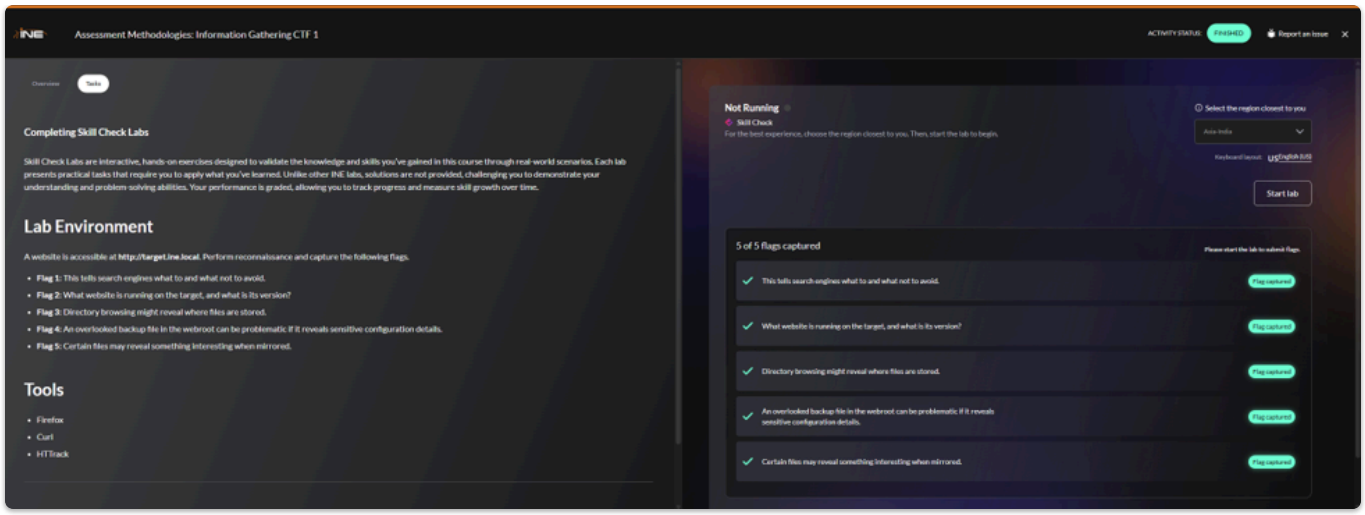
The output reveals the location of the final flag.

Screenshot



Flag 5 Discovery

Conclusion



By combining:

- Basic web reconnaissance
- Service enumeration
- Directory brute-forcing
- Backup file discovery
- Website mirroring

all five flags can be successfully identified without requiring any advanced exploitation techniques.

This challenge serves as an excellent introduction to the enumeration and reconnaissance techniques commonly used during penetration testing engagements.